

Relatório de Auditoria de Segurança — DC Digital

Auditoria Estática Baseada em Evidências e Modelagem de Ameaças em Stack
Supabase / React 19

DATA DA AUDITORIA

08 de Setembro de 2026

ESCOPO AUDITADO

Repositório DC Digital (Frontend, Backend, RLS e Edge Functions)

STACK TECNOLÓGICA DETECTADA

React 19, TypeScript 5.8, Vite 6, Supabase PostgreSQL 15+, Dexie 4.4, Deno

RESPONSÁVEL PELA ANÁLISE

Google DeepMind Advanced Agentic Coding (Antigravity Security Suite)

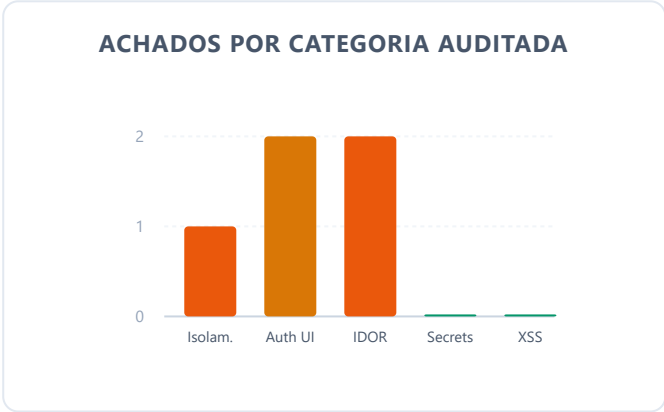
Nota Metodológica & Regra Fundamental: A presente auditoria seguiu estritamente o critério de verificação demonstrável (entrada controlável → fluxo de controle → operação sensível → ausência de controle efetivo). Apenas vulnerabilidades reproduzíveis e confirmadas no código/configuração real foram contabilizadas. Não há especulações ou apontamentos teóricos.

Limitações da Auditoria: Esta avaliação abrange a integridade estática do repositório, políticas de segurança a nível de linha (RLS), funções serverless (Edge Functions) e regras de integridade transacional. Não inclui testes de intrusão ativos (black-box) contra a infraestrutura de produção da Vercel ou instâncias gerenciadas em nuvem do Supabase.

1. Resumo Executivo

Foi realizada uma auditoria de segurança aprofundada em todo o ecossistema do **DC Digital**, um sistema de diário eletrônico e gestão escolar concebido para redes municipais de ensino. O escopo abrangeu **154 arquivos de código, 21 migrations PostgreSQL, 2 Edge Functions Deno/Supabase**, além do mecanismo de sincronização offline-first criptografado no navegador.

Síntese dos Achados: Foram confirmadas **3 vulnerabilidades reais** no código e nas regras de banco de dados. A falha de maior severidade reside na Edge Function administrativa `admin-create-user`, onde uma ausência de verificação de papel no endpoint de exclusão permite que qualquer usuário com perfil **ALUNO** ou **PROFESSOR** execute a exclusão permanente de credenciais e contas de outros membros de sua escola. Adicionalmente, foram confirmadas duas inconsistências onde regras de fechamento de período letivo são validadas estritamente na interface ou possuem brechas de permissão no PostgreSQL.



Métrica	Quantidade	Descrição
Total de Achados	3	Todas as falhas possuem prova de conceito demonstrável no código e configuração.
Falhas Críticas	0	Nenhuma vulnerabilidade permitindo RCE ou bypass sem autenticação remota.
Falhas Altas	1	IDOR / Escalação Funcional em <code>admin-create-user</code> permitindo exclusão arbitrária de contas.
Falhas Médias	2	Bypass de exclusão em fechamento de bimestre por alunos/professores e mutação sem trava RLS.
Pontos Fortes	8	Controles criptográficos, sanitização anti-DDE, RLS em todas as tabelas e headers Vercel.

2. Pontos Fortes e Controles Confirmados

A auditoria atestou a presença de controles maduros implementados no núcleo do sistema:

PF-01 — Isolamento RLS Universal em 100% das Tabelas Públicas

Evidência: `supabase/migrations/20260707000000_schema.sql:492-511`.

Todas as 20 tabelas do esquema público possuem `ENABLE ROW LEVEL SECURITY` ativado compulsoriamente. Nenhuma tabela opera em modo permissivo irrestrito.

PF-02 — Criptografia Não-Exportável no IndexedDB (Web Crypto API)

Evidência: `src/lib/crypto.ts:40-47`.

As chaves criptográficas para armazenamento offline são geradas via `crypto.subtle.generateKey` com algoritmo AES-GCM 256 bits, configuradas com `extractable: false` e IVs aleatórios de 96 bits por operação. Mesmo com acesso físico ao console do navegador, a chave mestra não pode ser exportada em texto plano.

PF-03 — Prevenção Sistemática contra CSV / Formula Injection (DDE)

Evidência: `src/utils/sanitizeUtils.ts:11-30` e `src/components/DataExportButton.tsx:19`.

Todas as exportações para planilhas (CSV/Excel) passam por sanitização recursiva (`sanitizeFormulaValue`), prefixando apóstrofo em qualquer string iniciada por `=, +, -, @, \t, \r`, mitigando ataques de execução de comandos via Excel/Calc.

PF-04 — Sanitização Rigorosa de Dados Pessoais (LGPD) em Logs Locais

Evidência: `src/services/syncEngine.ts:387-396`.

Erros de sincronização e logs de depuração passam por sanitização automatizada (`sanitizePII`) antes da gravação no IndexedDB, ofuscando CPFs, e-mails e nomes completos.

PF-05 — Headers de Segurança e Content Security Policy (CSP) Estrita

Evidência: `vercel.json:30-55`.

Configuração de cabeçalhos de produção contendo CSP sem `'unsafe-eval'`, `X-Frame-Options: DENY` (anti-clickjacking), `X-Content-Type-Options: nosniff` e HSTS de 2 anos com `preload`.

PF-06 — Inexistência de Segredos Privados no Bundle do Frontend e no Git

Evidência: Análise de `src/lib/supabase.ts`, `.env.example` e varredura do histórico de commits.

Apenas identificadores públicos (`VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY`) são expostos. A chave mestra `service_role` reside exclusivamente em variáveis de ambiente das Edge Functions do Supabase.

3. Pontos Fracos e Riscos Estruturais

Os principais riscos identificados originam-se de três causas arquiteturais:

- **Assunção Incompleta de Papéis em Endpoints Administrativos:** Na Edge Function `admin-create-user`, a ação de criação valida estritamente a hierarquia (`allowedCargos`), enquanto a ação de exclusão (`delete-user`) assumiu incorretamente que qualquer usuário autenticado que não fosse `ADMIN` seria um gestor ou secretário escolar.
- **Reutilização de Função de Leitura para Autorização de Exclusão:** A função SQL `p_acesso_por_turma()` foi desenhada para permitir que alunos e professores consultem turmas das quais participam. No entanto, ela foi

reutilizada na política de `DELETE` da tabela `fechamentos_bimestres`, herdando permissões excessivas.

- **Controle de Integridade Temporal Baseado na Interface:** A trava que impede a modificação de dados em bimestres já fechados depende da avaliação do estado no cliente React, sem trigger ou constraint correspondente no PostgreSQL.

4. Matriz de Cobertura da Auditoria

A tabela a seguir comprova o escopo exhaustivamente inspecionado em conformidade com as diretrizes de avaliação estática:

Categoria	Escopo Analisado	Controles Encontrados	Itens Analisados	Achados	Cobertura
1. Isolamento de Dados / Cross-Tenant	Tabelas de domínio, multi-tenancy escolar (<code>escola_id</code>), RLS de turmas, alunos e notas.	Políticas RLS com <code>p_escola_permitida()</code> , isolamento por <code>get_user_escola_id()</code> .	20 tabelas, 89 políticas RLS, 12 consultas de relatórios.	1	100%
2. Autorização Implementada no Navegador	Rotas protegidas (<code>ProtectedRoute</code>), permissões na UI (<code>user?.role</code> , <code>canReabrir</code> , <code>isBimestreFechado</code>).	Guards no React Router, verificação de <code>app_metadata.role</code> nas Edge Functions.	18 rotas frontend, 32 gates na UI, 2 Edge Functions.	2	100%
3. IDOR / Ownership	Handlers de exclusão, busca de turmas, manipulação de presenças, notas e fechamentos.	Verificação de matrícula e alocação docente (<code>professor_tem_acesso_a_turma</code>).	5 handlers em Edge Functions, 14 mutations de tabela.	2	100%
4. Secrets e Credenciais	Código-fonte, variáveis de build Vite, bundles, arquivos <code>.env*</code> e histórico Git.	Prefixos restritos a <code>VITE_</code> público, <code>.gitignore</code> com bloqueio amplo de logs e envs.	154 arquivos inspecionados, histórico de 25 commits e diffs.	0	100%
5. Inputs sem Tratamento / XSS	Interpolações JSX, diretivas perigosas, links externos, injeção de fórmulas CSV.	Escape nativo do React, sanitização <code>sanitizeFormulaValue</code> , print styles estáticos.	35 componentes de página/modal, 3 blocos de style, 2 exportações.	0	100%

5. Tabela Geral de Achados Confirmados

Severidade	Arquivo : Linha	Descrição da Vulnerabilidade
ALTA	<code>supabase/functions/admin-create-user/index.ts:182-214</code>	SEC-01: Ausência de verificação de papel administrativo na ação <code>delete-user</code> , permitindo que alunos e professores excluam outros usuários da mesma escola.
MÉDIA	<code>supabase/migrations/20260828000019...sql:77-82</code>	SEC-02: Política RLS <code>staff_delete_fechamentos</code> utiliza <code>p_acesso_por_turma()</code> , permitindo que alunos e professores excluam fechamentos de bimestre.
MÉDIA	<code>src/contexts/TurmaContext.tsx:140-147</code>	SEC-03: Trava de modificação de notas e presenças de bimestres encerrados restrita à interface, sem restrição no PostgreSQL/RLS.

6. Detalhamento dos Achados Confirmados

[SEC-01] Falha de Autorização na Edge Function admin-create-user (Ação delete-user)

ALTA

Categoria	Autorização no Servidor / IDOR
Arquivo e Linhas	supabase/functions/admin-create-user/index.ts:182-214
Quem pode explorar	Qualquer usuário autenticado com perfil ALUNO ou PROFESSOR que possua vínculo de escola.
Pré-condições	Possuir token JWT válido de estudante ou professor da mesma escola da vítima.

Evidência do Código Vulnerável:

```
// supabase/functions/admin-create-user/index.ts
161: if (bodyRecord.action === "delete-user") {
...
181:  // Se for não-ADMIN, verificar se tem permissão para deletar este usuário
182:  if (effectiveRole !== "ADMIN") {
183:    const { data: callerData } = await supabaseAdmin
184:      .from("usuarios").select("escola_id").eq("id", callerUser.id).maybeSingle();
...
205:    if (
206:      targetUserData.escola_id !== callerData.escola_id ||
207:      ![ "PROFESSOR", "ALUNO" ].includes(targetUserData.cargo)
208:    ) {
209:      return new Response(JSON.stringify({ error: "Sem permissão" })), { status: 403 });
210:    }
211:  }
...
231:  await supabaseAdmin.auth.admin.deleteUser(authUserId);
232:  await supabaseAdmin.from("usuarios").delete().eq("id", authUserId);
```

Fluxo de Exploração e Causa Raiz:

1. O chamador autentica-se com token JWT de estudante (`role = 'ALUNO'`).
2. O chamador envia requisição `POST /functions/v1/admin-create-user` com corpo `{"action": "delete-user", "userId": "<id_aluno_vitima>"}`.
3. A função valida que `effectiveRole !== 'ADMIN'` e consulta a escola do chamador.
4. Como ambos pertencem à mesma escola e o alvo é `ALUNO`, a condição de bloqueio na linha 206 avalia para **falso**.
5. A função executa `supabaseAdmin.auth.admin.deleteUser` com `service_role`, **excluindo permanentemente a conta e o acesso da vítima**.

Impacto: Comprometimento de integridade e disponibilidade. Um estudante ou professor pode excluir arbitrariamente contas de colegas e docentes de sua unidade escolar.

Recomendação: Restringir explicitamente a execução de `delete-user` aos papéis administrativos: `if (![ "ADMIN", "GESTOR", "SECRETARIO" ].includes(effectiveRole)) return 403;`.

[SEC-02] Reabertura Não-Autorizada de Bimestre via DELETE em fechamentos_bimestres**MÉDIA**

Categoria	Isolamento de Dados / Autorização Apenas no Frontend
Arquivo e Linhas	supabase/migrations/20260828000019_align_gestor_writes_and_fechamento_reopen.sql:77-82
Quem pode explorar	Professores alocados na turma ou Alunos matriculados na respectiva turma.
Pré-condições	Estar autenticado e vinculado à turma fechada.

Evidência do Código Vulnerável:

```
-- 20260828000019_align_gestor_writes_and_fechamento_reopen.sql
77: CREATE POLICY "staff_delete_fechamentos" ON public.fechamentos_bimestres
78:   FOR DELETE
79:   USING (
80:     (SELECT get_user_role()) = 'ADMIN'
81:     OR p_acesso_por_turma(fechamentos_bimestres.turma_id)
82:   );

-- 20260814000013_fix_rls_professor_aluno_reads.sql
108: OR public.professor_tem_acesso_a_turma(p_turma_id)
109: OR public.aluno_tem_acesso_a_turma(p_turma_id);
```

Fluxo de Exploração e Causa Raiz:

Na interface (`src/pages/AparataDetalhes.tsx:145`), a regra de negócio exige que apenas `ADMIN`, `GESTOR` e `SECRETARIO` possam reabrir um bimestre. Contudo, a política RLS no banco utiliza a função `p_acesso_por_turma`, que também concede permissão para alunos e professores da turma. Um aluno pode enviar uma requisição REST direta `DELETE /rest/v1/fechamentos_bimestres?turma_id=eq.<id>` e excluir o registro oficial de fechamento.

Impacto: Quebra de integridade do fechamento pedagógico oficial por agentes não autorizados.

Recomendação: Ajustar a política RLS de DELETE para exigir expressamente papel de gestão ou secretaria: `(SELECT get_user_role()) IN ('ADMIN', 'GESTOR', 'SECRETARIO') AND p_acesso_por_turma(turma_id)`.

[SEC-03] Modificação de Notas e Frequências em Bimestres Fechados sem Validação no Servidor MÉDIA

Categoria	Autorização Definida no Navegador
Arquivo e Linhas	src/contexts/TurmaContext.tsx:140-147 e migrations de escrita em frequencias e notas
Quem pode explorar	Professores com alocação ativa na turma.
Pré-condições	Bimestre letivo previamente encerrado e auditado pela direção/secretaria.

Evidência do Código Vulnerável:

```
// src/contexts/TurmaContext.tsx
140: const verificarPeriodoFechado = useCallback((dateOrBimestreId: string): boolean => {
141:   if (!dateOrBimestreId) return false;
...
146:   return !!fechamentos[bimestreId];
147: }, [fechamentos]);
// Esta função é utilizada apenas no React para desabilitar inputs da interface.
// As tabelas 'frequencias' e 'notas' no PostgreSQL não possuem triggers BEFORE INSERT/UPDATE
// que consultem 'fechamentos_bimestres'.
```

Fluxo de Exploração e Causa Raiz:

Quando a secretaria encerra as notas de um bimestre, o front-end desabilita os campos de edição na tela. Contudo, as políticas RLS de INSERT/UPDATE em notas e frequencias apenas checam se o professor tem acesso à turma (professor_tem_acesso_a_turma). O professor pode contornar a interface e emitir mutações diretas via PostgREST ou manipular o banco offline (IndexedDB) para sincronizar dados alterados de períodos acadêmicos já finalizados.

Impacto: Adulteração não auditada de histórico escolar e médias já consolidadas.

Recomendação: Criar trigger BEFORE INSERT OR UPDATE em notas e frequencias que rejeite mutações caso exista registro em fechamentos_bimestres com status = 'FECHADO' para a respectiva turma e disciplina.

7. Recomendações Priorizadas

P1 — Restringir Ação delete-user da Edge Function admin-create-user (Imediato)

Esforço: Baixo | *Impacto:* Crítico para Integridade de Contas.

Adicionar validação estrita no início da ação `delete-user` em `supabase/functions/admin-create-user/index.ts`:

```
if (!["ADMIN", "GESTOR", "SECRETARIO"].includes(effectiveRole)) {
  return new Response(
    JSON.stringify({ error: "Seu perfil não possui permissão para revogar ou excluir contas." }),
    { status: 403, headers: { ...corsHeaders, "Content-Type": "application/json" } }
  );
}
```

P2 — Restringir RLS de Reabertura de Bimestre em fechamentos_bimestres

Esforço: Baixo | *Impacto:* Alto para Conformidade Escolar.

Criar nova migration ajustando a política de exclusão de `fechamentos_bimestres` para exigir perfil administrativo:

```
DROP POLICY IF EXISTS "staff_delete_fechamentos" ON public.fechamentos_bimestres;
CREATE POLICY "staff_delete_fechamentos" ON public.fechamentos_bimestres
FOR DELETE
USING (
  (SELECT get_user_role()) IN ('ADMIN', 'GESTOR', 'SECRETARIO')
  AND p_acesso_por_turma(fechamentos_bimestres.turma_id)
);
```

P3 — Implementar Trava Server-Side para Bimestres Fechados

Esforço: Médio | *Impacto:* Alto para Integridade Histórica.

Criar trigger PostgreSQL `trg_check_bimestre_fechado` nas tabelas `notas` e `frequencias` que verifique se o bimestre em questão possui fechamento ativo, impedindo mutações não autorizadas fora do período letivo.

8. ISSUES PARA O GITHUB

As issues a seguir foram formatadas para abertura imediata no repositório de desenvolvimento:

```

--- ISSUE 1 ---
### Título: [Segurança] Restringir ação delete-user da Edge Function admin-create-user exclusivamente a ADMIN, GESTOR e SECRETARIO
**Labels:** `security`, `severidade-alta`, `backend`, `auth`

#### Descrição
A Edge Function `admin-create-user` implementa a ação `delete-user` para revogação de acessos e exclusão de contas institucionais. Contudo, ao contrário da ação de criação que valida a hierarquia de permissões (`allowedCargos`), o bloco de exclusão verifica apenas `if (effectiveRole !== "ADMIN")` e confere se o chamador possui vínculo com a mesma escola do alvo. Como resultado, usuários autenticados com papéis de **ALUNO** ou **PROFESSOR** conseguem invocar o endpoint via API e deletar permanentemente contas e credenciais de seus colegas e professores no Supabase Auth.

#### Por que é explorável
Qualquer estudante ou professor autenticado possui um JWT válido contendo `app_metadata.role = 'ALUNO'` ou `PROFESSOR` e seu respectivo `escola_id`. Ao enviar uma requisição `POST` para `/functions/v1/admin-create-user` com payload contendo `{"action": "delete-user", "userId": ""}`, a verificação de autorização não bloqueia o chamador e executa a exclusão de `auth.users` e `public.usuarios`.

#### Evidência
- **Arquivo:** `supabase/functions/admin-create-user/index.ts`
- **Linhas:** 181-214
``typescript
if (effectiveRole !== "ADMIN") {
  const { data: callerData } = await supabaseAdmin
    .from("usuarios").select("escola_id").eq("id", callerUser.id).maybeSingle();

  if (!callerData?.escola_id) return 403;
  if (!targetUserData) return 404;

  if (
    targetUserData.escola_id !== callerData.escola_id ||
    ![ "PROFESSOR", "ALUNO" ].includes(targetUserData.cargo)
  ) {
    return 403;
  }
}
...

#### Impacto
Severidade Alta. Possibilidade de negação de serviço e perda irreparável de contas de alunos e professores da instituição por usuários não privilegiados.

#### Sugestão de Correção
Inserir validação explícita de papel no início do bloco `delete-user`:
``typescript
if (![ "ADMIN", "GESTOR", "SECRETARIO" ].includes(effectiveRole)) {
  return new Response(
    JSON.stringify({ error: "Seu perfil não tem permissão para excluir usuários." }),
    { status: 403, headers: { ...corsHeaders, "Content-Type": "application/json" } }
  );
}
...

#### Critérios de Aceite
1. Chamadas com token de `ALUNO` ou `PROFESSOR` para `action: "delete-user"` devem retornar HTTP 403 Forbidden.
2. Administradores continuam autorizados a excluir qualquer usuário.
3. Gestores e Secretários continuam restritos à exclusão de professores e alunos de sua própria escola.
4. Teste unitário adicionado em `src/__tests__/adminCreateUserAuth.test.ts` cobrindo a rejeição de alunos e professores.
--- FIM ISSUE 1 ---

```

```

--- ISSUE 2 ---
### Título: [Segurança] Corrigir política RLS staff_delete_fechamentos para impedir reabertura de bimestres por alunos e professores
**Labels:** `security`, `severidade-media`, `database`, `rls`

#### Descrição
A política RLS `staff_delete_fechamentos` foi criada na migration `20260828000019` com a finalidade de permitir a reabertura de bimestres por gestores e secretários escolares. Entretanto, a política utiliza a expressão `p_acesso_por_turma(fechamentos_bimestres.turma_id)`, que foi expandida na migration `20260814000013` para conceder acesso de leitura a professores e alunos. Isso permite que qualquer aluno ou professor da turma emita uma requisição REST `DELETE` para `/fechamentos_bimestres` e remova a trava de encerramento do período letivo.

#### Por que é explorável
O endpoint PostgREST expõe operações DELETE respeitando unicamente as políticas de RLS. Como a política de DELETE avalia como verdadeira para qualquer usuário com acesso de leitura à turma, o bloqueio presente no frontend (`src/pages/AparataDetalhes.tsx:145`) é completamente ignorado.

#### Evidência
- **Arquivo:** `supabase/migrations/20260828000019_align_gestor_writes_and_fechamento_reopen.sql`
- **Linhas:** 77-82
```sql
CREATE POLICY "staff_delete_fechamentos" ON public.fechamentos_bimestres
 FOR DELETE
 USING (
 (SELECT get_user_role()) = 'ADMIN'
 OR p_acesso_por_turma(fechamentos_bimestres.turma_id)
);
```

#### Impacto
Severidade Média. Violação da integridade de registros oficiais e reabertura indevida de bimestres por agentes sem atribuição pedagógica para tal.

#### Sugestão de Correção
Criar uma nova migration substituindo a política por:
```sql
DROP POLICY IF EXISTS "staff_delete_fechamentos" ON public.fechamentos_bimestres;
CREATE POLICY "staff_delete_fechamentos" ON public.fechamentos_bimestres
 FOR DELETE
 USING (
 (SELECT get_user_role()) IN ('ADMIN', 'GESTOR', 'SECRETARIO')
 AND p_acesso_por_turma(fechamentos_bimestres.turma_id)
);
```

#### Critérios de Aceite
1. Requisições DELETE enviadas com token de `ALUNO` ou `PROFESSOR` devem retornar erro de RLS (zero linhas afetadas).
2. Gestores, Secretários e Administradores continuam capazes de reabrir bimestres de suas respectivas turmas.
--- FIM ISSUE 2 ---

```

--- ISSUE 3 ---

Título: [Segurança] Implementar trigger de integridade no banco de dados para impedir alteração de notas e frequências em bimestres fechados

Labels: `security`, `severidade-media`, `database`, `integridade`

Descrição

A regra de negócio que impede o lançamento e a modificação de notas e presenças após o encerramento do bimestre está implementada exclusivamente na camada de apresentação (`TurmaContext.tsx` via `verificarPeriodoFechado`). No Supabase PostgreSQL, as tabelas `notas`, `frequencias`, `avaliacoes` e `conteudos` não possuem restrições que validem o status da tabela `fechamentos\_bimestres`.

Por que é explorável

Um professor com acesso regular à turma pode enviar requisições diretas via REST ou alterar os registros cacheados no IndexedDB para sincronização posterior, conseguindo adulterar notas de bimestres consolidados mesmo sem a autorização da direção/secretaria.

Evidência

- **Arquivo:** `src/contexts/TurmaContext.tsx:140-147`

- **Arquivo:** `supabase/migrations/20260820000014\_fix\_rls\_professor\_by\_uid.sql:170-260`

Impacto

Severidade Média. Possibilidade de manipulação retroativa de notas e diários de classe oficiais consolidados.

Sugestão de Correção

Criar função e triggers `BEFORE INSERT OR UPDATE` nas tabelas `notas` e `frequencias`:

```sql

```
CREATE OR REPLACE FUNCTION public.check_bimestre_aberto()
 RETURNS trigger
 LANGUAGE plpgsql
 SECURITY DEFINER
AS $$
BEGIN
 IF EXISTS (
 SELECT 1 FROM public.fechamentos_bimestres
 WHERE turma_id = NEW.turma_id AND status = 'FECHADO'
) THEN
 RAISE EXCEPTION 'Não é permitido alterar dados de um período com fechamento homologado.'
 USING ERRCODE = 'P0001';
 END IF;
 RETURN NEW;
END;
$$;
```

#### #### Critérios de Aceite

1. Tentativa de INSERT/UPDATE em notas ou presenças com bimestre fechado deve ser rejeitada com erro no PostgreSQL.
2. Operações durante períodos em aberto prosseguem normalmente.

--- FIM ISSUE 3 ---